AI/ML Written Report:

Word count: 2477

Abstract:

The implementation of deep learning techniques in neuroimaging holds considerable promise for the early diagnosis of dementia. However, models frequently struggle with extracting subtle structural features from MRI scans. This study evaluates the performance of three convolutional neural network (CNN) architectures, each built to address the shortcomings of the last. These models classified varying stages of dementia (Mild, Moderate, and No Impairment) from an imbalanced dataset of 2D MRI scans. The diagnostic efficacy of a baseline CNN (Model 1) and an augmented CNN (Model 2) was compared against a pre-trained ResNet50 model utilising transfer learning (Model 3). Due to the skewed nature of the dataset, macro-averaged metrics and per-class analyses were used to gauge true diagnostic performance. The ResNet50 architecture proved to be the superior classifier, achieving an overall accuracy of 81% and a macro-average F1-score of 0.78. It addressed the prediction failures of the minority class seen in the baseline model, which experienced a deceptive ROC-AUC paradox that masked a poor Moderate Impairment recall of 0.50. It also overcame the spatial degradation caused by the augmentation of the second model. These findings indicate that transfer learning can effectively map biological markers associated with dementia. For safe clinical implementation, future work should focus on 3D volumetric architectures, targeted oversampling of minority classes, and explore use-cases for Explainable AI to ensure diagnostic transparency.

Introduction:

Major Neurocognitive Disorder, commonly known as dementia, is the persistent and progressive decline in at least one cognitive domain, such as executive function, learning, or memory (Emmady et al., 2022). A defining clinical feature is that impairment must interfere with daily functioning and independence. While dementia and Alzheimer's disease are often described synonymously, with ~70% of dementia being Alzheimer's-induced dementia, dementia is a broad umbrella term encompassing multiple neurodegenerative disorders, including vascular dementia, Lewy body disease and Parkinson's disease. Dementia is highly prevalent in the UK (~850,000 cases as of 2014) and is estimated to triple by 2050 (Hazan et al., 2024). In response to this public health challenge, the NHS uses the Dementia Diagnosis Rate (DDR) as a key quality metric, aiming to diagnose two-thirds of those living with dementia. This demonstrates that there is an urgent need to use more efficient and accurate methods to diagnose dementia.

Magnetic Resonance Imaging (MRI) is a non-invasive imaging technique which is widely used to image abnormal brain tissues (Ghadimi and Thomas, 2025; Krishnapriya and Karuna, 2023). It excels in producing exceptional soft tissue contrast and by using magnetism, it is safer than CT (computed tomography), which utilises ionising radiation. Although MRIs produce high-

resolution images which contain an abundance of structural information, it is time-consuming and expensive to manually evaluate the mammoth volumes of data generated. Machine learning (ML) has the potential to revolutionise healthcare by rapidly identifying unseen patterns in MRI images such as tumours and the characteristic atrophy seen in dementia (Ahsan et al., 2022). However, it is essential that predictions made by ML models are accurate and unbiased, as they directly impact patient lives (Faiyazuddin et al., 2025).

The aim of this project was to design a machine learning model capable of precisely classifying MRI images into 3 categories: No Impairment, Mild Impairment and Moderate Impairment. This report is split into 5 sections. The Methods section will describe the iterative process undertaken to design our final model. Then, the Results will highlight the key outputs produced. The Discussion will explore the results and their implications. Finally, the section for ethical implications will discuss the ethics of this project and how artificial intelligence (AI) and ML fits into healthcare, before concluding the paper.

Methods:

The dataset was comprised of 5076 images of the brain taken via MRI. 2560 (50.4%) of these were MRIs of brains with no impairment. 1792 (35.3%) were mild impairment. 724 (14.3%) were moderate impairment. This represents a highly skewed and imbalanced dataset, where over half of all images were no impairment. This mean that the classifier has a higher likelihood to select NoImpairment, since it statistically would be correct half the time. Figure 1 provides a graph showing the distribution of images.

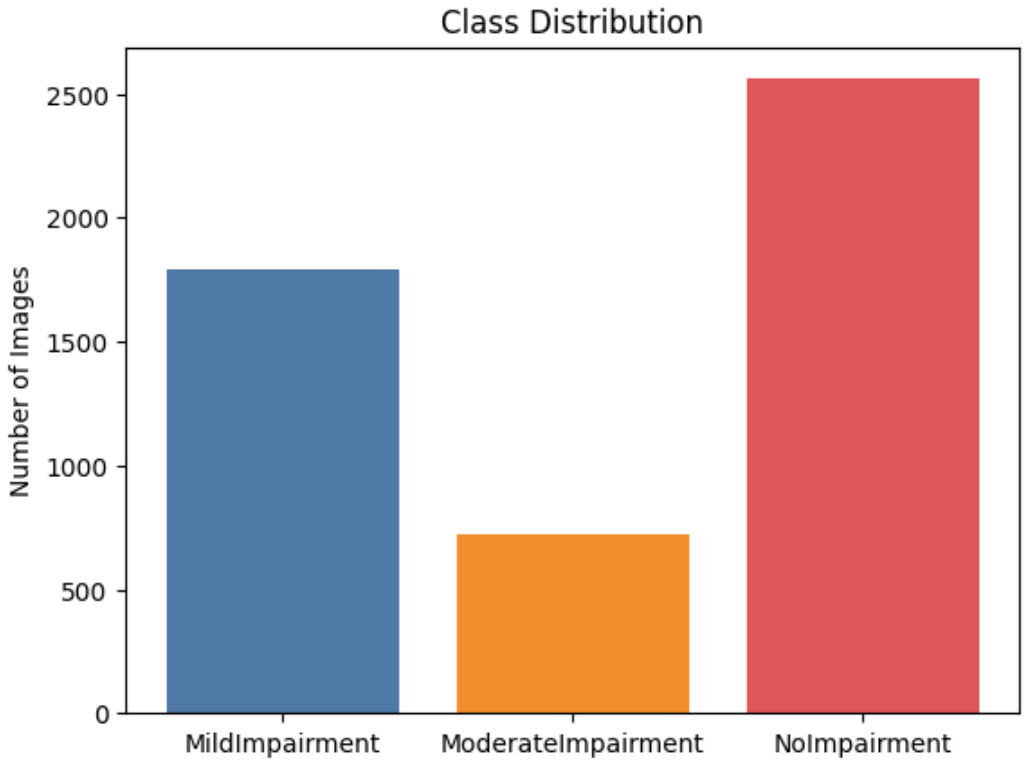


Figure 1: A bar chart showing the distribution of Mild, Moderate and No impairment in the images in the dataset.

The training, test and validation datasets were created using an 80 /10 /10 split using *keras.utils.image_dataset_from_directory*. This function streams the images in memory-efficient batches of 32, utilising a fixed random seed (2614) to ensure reproducible class shuffling. Image size was standardised to 256 x 256 for all images to ensure dimensional consistency. To evaluate the model's ability to extract features from raw medical images, no further pre-processing steps were applied (such as cropping or CLAHE). This minimised the risk of removing relevant boundary information and served as a baseline for model performance.

Three convolutional neural networks (CNN) architectures were evaluated to address the research question, chosen for their established efficacy in computer vision and image classification (Sharma et al., 2018). CNNs are deep learning algorithms capable of classifying objects (Figure 2).

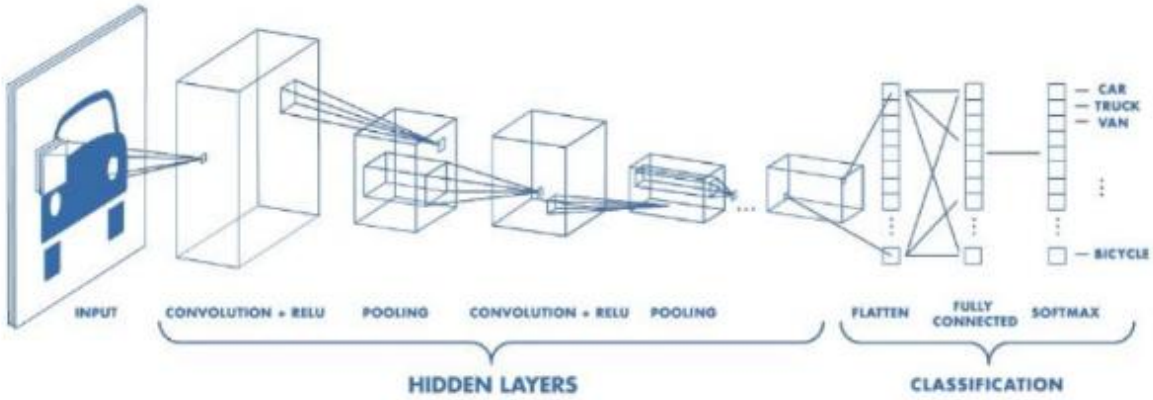


Figure 2: A diagram illustrating the architecture of a CNN. An input image is passed a series of hidden layer consisting of convolution, Rectified Linear Unit (ReLU) activation and pooling operations to extract features. The output is then flattened through fully connected (or Dense) layers before going through a SoftMax layer which provides classification probabilities for each of the categories (Purwono et al., 2023).

Model 1:

Model 1 was used as a baseline. It features 1 convolutional layer (32 filters, ReLU activation, 3x3 kernel) and a MaxPooling layer (2x2), while incorporating a dropout of 0.1 to mitigate overfitting. This model was compiled by *Adam* (learning rate = 0.003), a stochastic gradient descent optimisation algorithm. Unlike a standard stochastic gradient descent algorithm which applies a fixed learning rate for all weight updates, Adam computes adaptive learning rates for individual parameters by estimating the first and second moments of the gradients (Kingma and Lei Ba, 2014; Robbins and Monro, 1951). This makes it memory efficient, robust to noise and require minimal hyperparameter tuning. The loss function was *sparse_categorical_crossentropy* because we had more than 2 groups.

Model 2:

To address the overfitting in Model 1, this architecture was deepened to include 3 sets of convolution and pooling, facilitating better feature extraction. Data augmentation was introduced to prevent the model from memorising the images and to diversify our narrow training set. This comprised of random horizontal flips (to retain biological plausibility), rotations (0.1) and zooms (0.2). Learning rate was dropped to 0.0001 for stable convergence. In addition, class weights were added to penalise misclassifications in less represented classes (Mild: 1.022, Moderate : 2.306, No : 0.630). All other hyperparameters remained the same as Model 1.

Model 3:

Driven by the insufficient depth of the previous model, we decided to leverage transfer learning. Transfer learning is a technique which uses the knowledge acquired from one task and applies it to a similar but distinct task to improve performance (Torrey and Shavlik, 2010). We used ResNet50, a 50-layer CNN. ResNet mitigates the vanishing gradient problem commonly seen in neural networks by using residual blocks with skip connections. This allows the gradients to bypass certain layers during backpropagation (Shree Charran and Dubey, 2022).

To accommodate this pre-trained model, we initially resized the greyscale images to 224 x 244 and converted them into 3-channel RGB format. Data augmentation was refined, with random horizontal flips and a slight 0.05 rotation. The ResNet50 base was initialised with ImageNet weights. Although ImageNet is comprised of natural images, its foundational convolutional layers act as edge and texture detectors, making it suitable for neuroimaging.

Most of the base model was frozen to retain its feature-extraction capabilities. However, the final five layers (excluding batch normalisation) were unfrozen to allow the model to fine-tune specifically to the MRI images. Batch normalisation was kept frozen to ensure the model uses the moving mean and variance statistics which were built from the ImageNet dataset. This prevented destabilisation and large gradient updates to be caused by the MRI data.

The extracted features passed into a fully connected layer (128 units, ReLU activation) before concluding into a 3-unit SoftMax classification layer. The model had a dropout rate of 0.2, a learning rate of 0.0001, and the same class weights seen in Model 2.

Results:

Model 1:

Figure 3 shows the accuracy and loss graphs created during training and validation of Model 1. While the training was set for 20 epochs, the early stopping callback (which monitored validation loss with a patience of 4 epochs) halted training at epoch 7. Model 1 achieved an overall accuracy of 0.71, with a macro-average F1-score of 0.65. However, per-class analysis highlighted the model's performance varied significantly,

with a F1-score of 0.54 in the moderate impairment group. In addition, the recall of the moderate impairment group was 0.50, revealing it missed half of those cases. Figure 4 shows the Receiver Operating Characteristic (ROC) curves for Model 1. The model displays strong discriminatory ability across all classes, with Area Under the Curve (AUC) values of 0.87 for Moderate Impairment, 0.86 for No Impairment, and 0.83 for Mild Impairment.

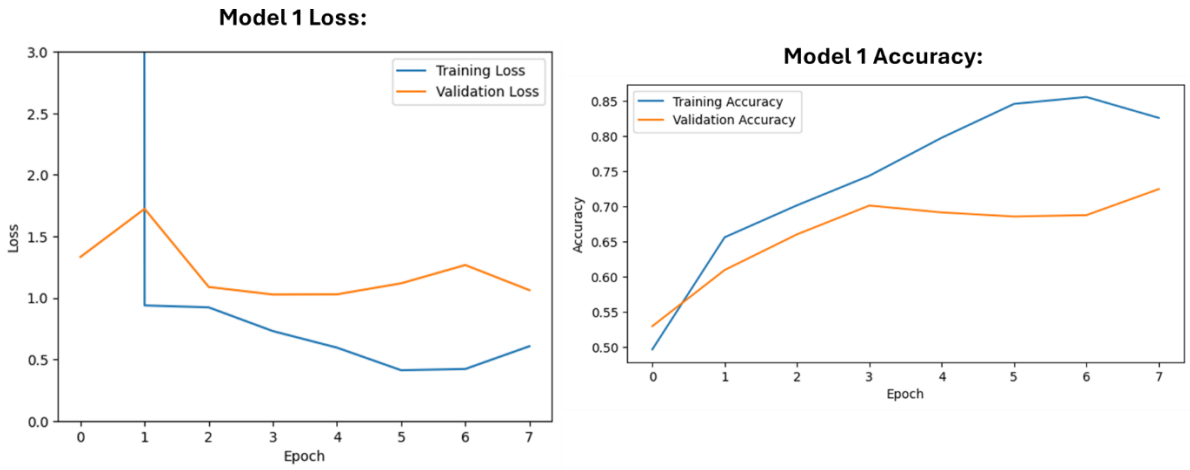


Figure 3: Accuracy and loss graphs for Model 1. The large gaps between the training and validation accuracy indicate severe overfitting.

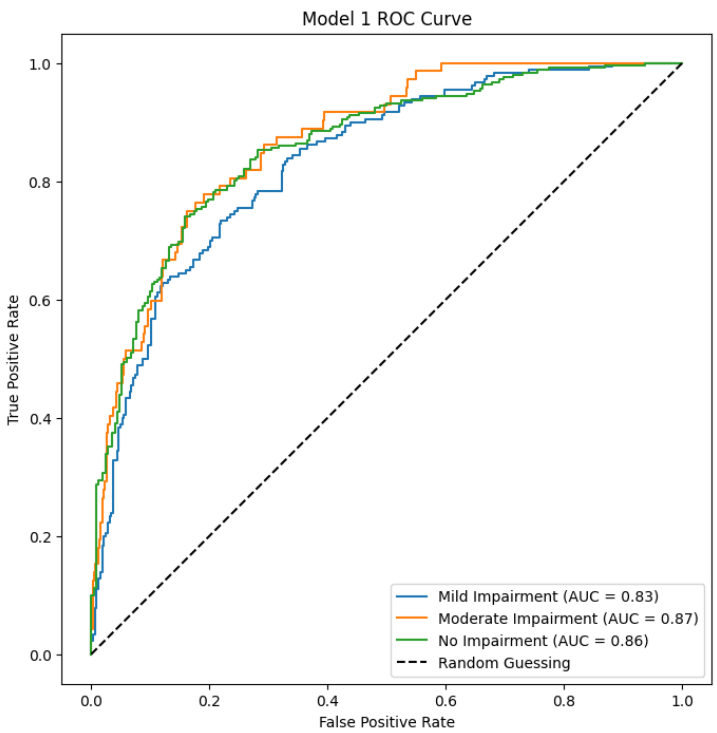


Figure 4: Receiver Operating Characteristic (ROC) curves evaluating the performance of Model 1. The graph illustrates the true positive rate (TPR) against the true negative rate (TNR) across the Mild, Moderate and No Impairment classes, alongside their Area Under the Curve (AUC) scores.

Model 2:

Model 2 demonstrated a clear degradation in overall performance, achieving an accuracy of 0.50 and a macro-average F1-score of 0.41. Figure 5 shows this clearly, with loss remaining high and stagnant throughout all 8 epochs and accuracy fluctuating erratically without convergence. Per-class analysis highlighted a critical inability to classify Mild Impairment (F1: 0.15) and an AUC of 0.36 as seen by Figure 6, indicating the model was worse than random guessing. In addition, Moderate Impairment saw a recall of 0.74 and a precision of 0.26, revealing a severe false-positive bias where the model consistently over-predicted the class.

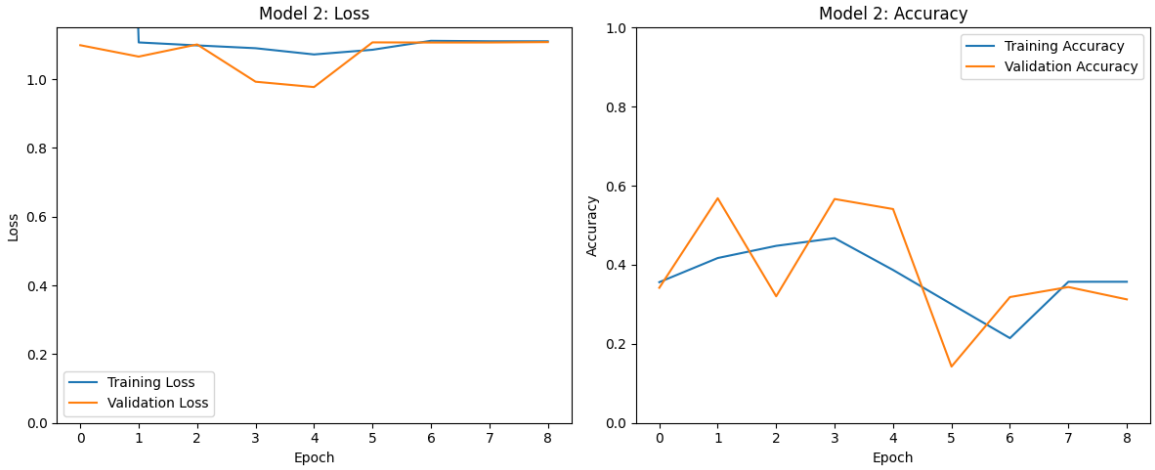


Figure 5: Accuracy and loss graphs for Model 2

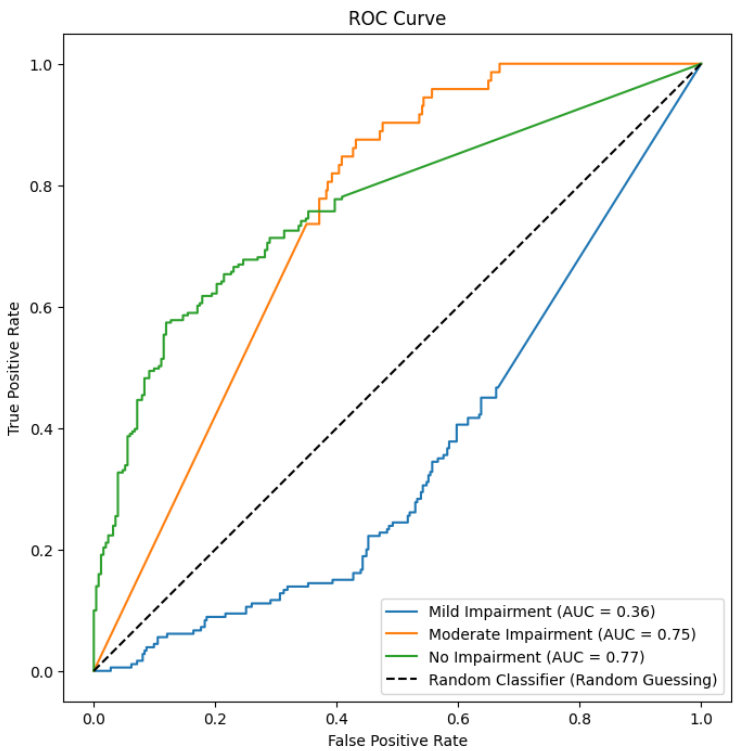


Figure 6: ROC curves evaluating the performance of Model 2. The graph illustrates the TPR against the TNR across the Mild, Moderate and No Impairment classes, alongside their AUC scores.

Model 3:

Model 3, utilising the pre-trained ResNet50 architecture, demonstrated substantial improvements in metrics across the board, with an overall accuracy of 0.81, and a macro-average F1-score of 0.78. Figure 7 suggests a highly stable learning process, with both training and validation loss consistently converging over 30 epochs without significant overfitting. Per-class analysis saw an F1-score of 0.79 for Mild Impairment, a significant improvement to Model 2. Our minority class, Moderate Impairment, also saw a strong precision of 0.81. The ROC curves in Figure 8 also confirm this improved performance, with near perfect AUC's closely hugging the perfect classifier threshold (Moderate: 0.96, Mild: 0.92, No: 0.92)

Figure 7: Accuracy and loss graphs for Model 3

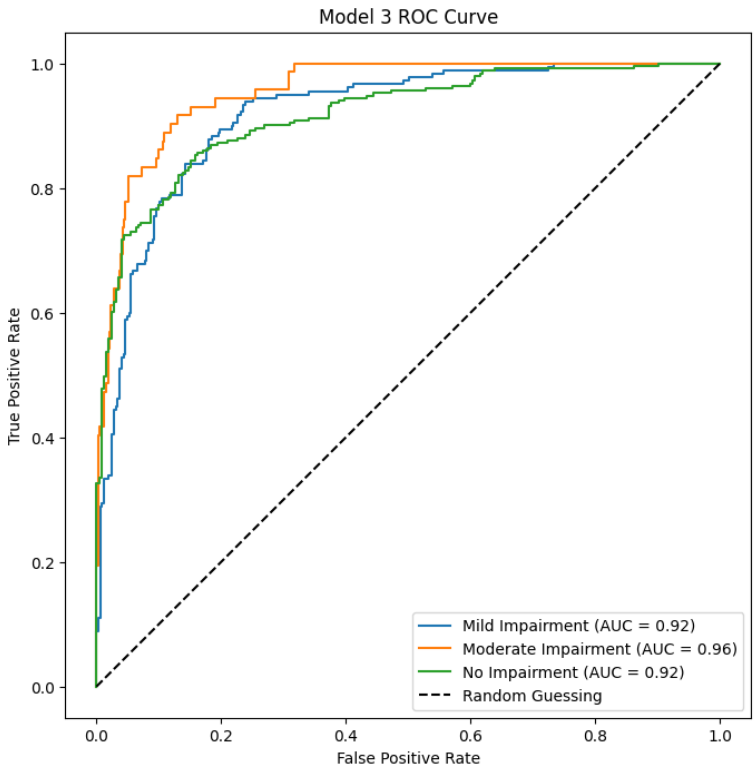


Figure 8: ROC curves evaluating the performance of Model 3. The graph illustrates the TPR against the TNR across the Mild, Moderate and No Impairment classes, alongside their AUC scores.

Comparative Results:

Table 1 shows the classification reports of all three models.

Model	Accuracy (%)	Recall (%)	Precision (%)	F1-score (%)
Model 1	**71**	**64**	**68**	**65**
Model 2	**50**	**51**	**45**	**41**
Model 3	**81**	**76**	**80**	**78**

Table 1: Comparative summary of overall performance metrics for all three models. To account for the severe class imbalance within the dataset, recall, precision and F1-scores are reported using their macro-average values. This ensures the minority classes (Mild and Moderate) are evaluated equally alongside the majority class (No Impairment).

Figure 9 shows micro-average ROC curves comparing the classification performance of all three models.

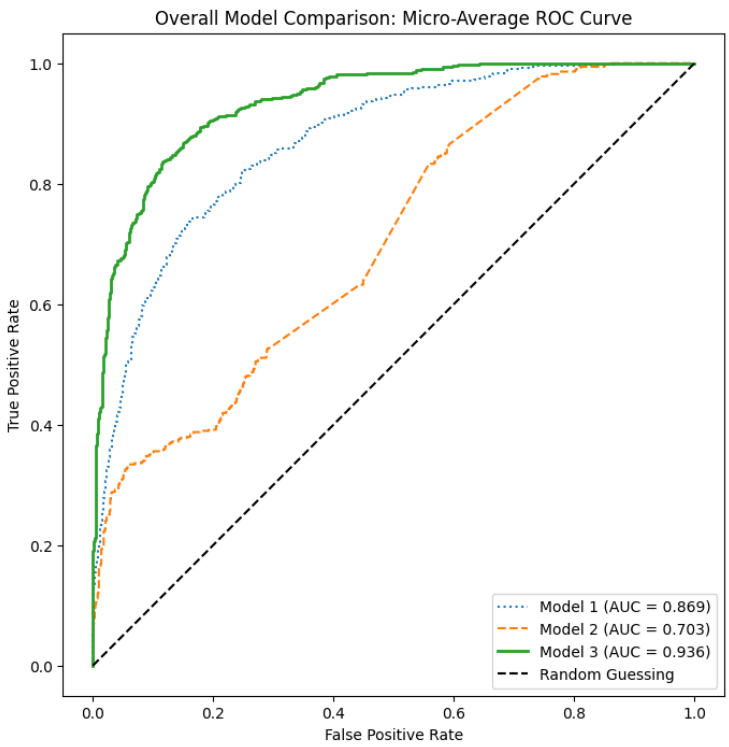


Figure 9: Micro-average ROC curves comparing the overall classification performance of all three models. The AUC values show Model 3 (AUC = 0.936) was the best at discriminating between the three classes, compared to the baseline Model 1 (AUC = 0.869) and augmented Model 2 (AUC = 0.703).

Figure 10 shows comparative confusion matrices across all three models. Model 3 had the strongest overall performance, achieving the highest correct classification rates across all three classes. In contrast, Model 1 struggled with identifying specifically moderate impairment, while Model 2 exhibited across the board misclassifications.

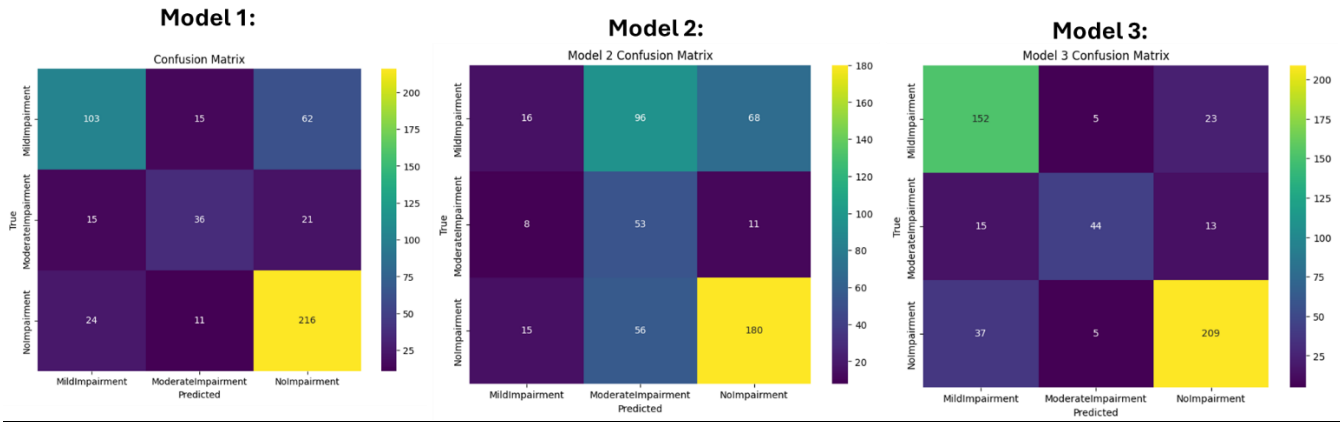


Figure 10: Comparative confusion matrices for all three models. They display the classification results for the three class: "MildImpairment" , "ModerateImpairment", and "NoImpairment". True labels are the rows and predicted labels on the columns, providing a visual tool to assess and compare classification performance.

Discussion:

The progression of models in this study highlights the challenges of extracting biological and structural markers from imbalanced medical imaging datasets. The ResNet50 architecture (Model 3) proved to be the superior classifier, achieving an overall accuracy of 81% and a macro-average F1-score of 0.78. By leveraging transfer learning, the model utilised pre-learned hierarchies to map the subtle patterns and intricacies associated with brain atrophy and thus dementia. This aligns with the available literature which demonstrates that transfer learning — including other models such as VGG16 (Naser and Deen, 2020) — can mitigate the data scarcity common in medical imaging (Warren and Moustafa, 2022).

On the other hand, Model 2 highlights the sensitivity of neuroimaging. The implementation of random zooms and rotations likely obscured or cropped essential biological markers of cortical atrophy. These subtle differences are required to distinguish between the highly similar No Impairment and Mild Impairment classes. This, coupled with the aggressive class weight of 2.306 applied to the Moderate Impairment group caused the model to overpredict the moderate class, resulting in high recall (0.74) but low precision (0.26). This stresses the importance of maintaining the brain's rigid spatial information; drastic augmentation can destroy the structural context needed for accurate diagnosis.

When evaluating imbalanced datasets, careful metric selection in paramount. Macro-averaging was used throughout this study to provide an unweighted performance assessment. This ensures that the majority class (No Impairment) does not inflate or bias the overall diagnostic performance.

A paradox appears regarding the Moderate Impairment class in Model 1, which achieved the highest AUC (0.87) despite having the lowest recall (0.50) and F1-score (0.54). This is driven by the severe class imbalance and its interaction with the One-vs-Rest calculation of the ROC curve. Because Moderate Impairment constitutes only 14.3% of the dataset, the remaining 85.7% of images form a large true negative class. As a result, the false positive rate ($FPR = \frac{FP}{(FP + TN)}$) can remain numerically small due to the dominating denominator, even when a large number of misclassifications occur. Consequently, the AUC may suggest a strong classifier even with suboptimal minority-class detection. Crucially, AUC assesses ranking performance at all thresholds, whereas recall and F1-score are determined at a specific decision boundary (by choosing the highest probability). The elevated AUC suggests that Moderate Impairment cases are more frequently assigned higher predicted probabilities than healthy images, potentially reflecting structural characteristics such as ventricular enlargement and cortical thinning. However, the low recall shows that at the specific decision boundary, the model fails to confidently predict half of true Moderate Impairment cases. This highlights the limitation on relying purely on ROC-AUC for imbalanced data and emphasises the importance of per-class analysis to prevent misrepresentation of performance and evaluate true diagnostic ability.

Limitations and Reflection:

The dataset's small and imbalanced nature was a critical limitation of this study. With a test set of only 503 images, this is a relatively small corpus for training deep convolutional

architectures. This inherently restricts the models' exposure to diverse structural variances and increases the risk of overfitting. In future works, targeted minority augmentation could be used to adjust the distribution of images between the classes, coupled alongside under sampling the majority class until an even distribution is reached prior to training. We believe this could be the crucial change needed to drive up performance metrics into 90%+, as seen by other papers (Ali et al., 2025; Amine and Mourad, 2025; Krishnapriya and Karuna, 2023).

A key limitation is the use of isolated 2D planar slices from a pooled public dataset, thus discarding the 3D volumetric context of the original MRI scans. This means the models cannot make predictions on the full three-dimensional biological context needed to conclusively understand and identify dementia. By making predictions on a 2D cross-section, the model diagnostic depth is restricted. Future iterations of this project should use patient-stratified, 3D MRI volumes and use 3D convolutional architectures. The efficacy of this approach is supported by the work of Chen et al. (2025), who utilised 3D-ResNet for the diagnosis of Alzheimer's disease, achieving 92.33% accuracy.

In future works, we also hope to implement Explainable AI techniques such as Grad-CAM heatmaps, which will allow us to spatially localise the features and confirm they come from biologically plausible regions (Selvaraju et al., 2019). This will help establish a more robust and clinically transparent tool.

Ethical Implications:

The integration of deep learning models into clinical practice has the potential to greatly enhance diagnostic efficiency and accuracy in the NHS. But it does come with important ethical considerations. The inherent "black-box" nature of convolutional architectures such as ResNet50 significantly limits transparency. If a misdiagnosis was to occur, where to place liability is a regulatory grey area. Privacy is also a key concern; patients must always consent to their information being used in the training of deep learning models and their data must be securely stored and maintained. In addition, although using an anonymised public dataset avoided privacy concerns in this study, it carries the risk of algorithmic bias. If the training data is not representative of the wider population, especially minorities and those historically underrepresented in healthcare, the model may perform inequitably across demographics and further exacerbate systemic healthcare disparities. Models used in healthcare must be morally grounded and have the regulatory framework and oversight to ensure patient safety, equitable access and clear accountability in clinical decision-making.

Conclusion:

This paper evaluated three deep learning architectures for classifying dementia stages from MRI images. The pre-trained ResNet50 architecture (Model 3) was the best classifier, achieving an 81% overall accuracy and a macro-average F1-score of 0.78. By leveraging transfer learning, this approach overcame the minority-class prediction failures and inadequate accuracy in the baseline and augmented models. This result highlights the potential of transfer learning to identify the biological markers of brain atrophy and thus

enhance early diagnosis and efficiency. Before implementing this framework as a robust clinical tool, future iterations of this project should address the technical and ethical limitations. Ensuring the model is trained on a well-sized, representative and balanced dataset, as well as integrating Explainable AI techniques will improve performance, assist in transparency and mitigate algorithmic bias.

References:

Ahsan, M. M. et al. (2022) Machine-Learning-Based Disease Diagnosis: A Comprehensive Review. Healthcare. [Online] 10 (3), 541. [online]. Available from: https://pmc.ncbi.nlm.nih.gov/articles/PMC8950225/ (Accessed 16 February 2026).

Ali, R. R. et al. (2025) Learning Architecture for Brain Tumor Classification Based on Deep Convolutional Neural Network: Classic and ResNet50. Diagnostics. [Online] 15 (5), 624. [online]. Available from: https://pmc.ncbi.nlm.nih.gov/articles/PMC11898819/ (Accessed 22 February 2026).

Chen, D. et al. (2025) MRI-based diagnostic model for Alzheimer's disease using 3D-ResNet. Biomedical Physics & Engineering Express. [Online] 11 (3), 035031. [online]. Available from: https://pubmed.ncbi.nlm.nih.gov/40354785/ (Accessed 22 February 2026).

Emmady, P. D. et al. (2022) Major Neurocognitive Disorder (Dementia) [online]. Available from: https://www.ncbi.nlm.nih.gov/books/NBK557444/ (Accessed 2 February 2026).

Faiyazuddin, Md. et al. (2025) The Impact of Artificial Intelligence on Healthcare: A Comprehensive Review of Advancements in Diagnostics, Treatment, and Operational Efficiency. Health Science Reports. [Online] 8 (1),. [online]. Available from: https://onlinelibrary.wiley.com/doi/pdf/10.1002/hsr2.70312 (Accessed 16 February 2026).

Hazan, J. et al. (2024) Dementia diagnosis rates and the impact of ethnicity, rurality and deprivation. Aging & Mental Health. [Online] 29 (1), 138–144. [online]. Available from: https://www.tandfonline.com/doi/full/10.1080/13607863.2024.2374936 (Accessed 16 February 2026).

Kingma, D. & Lei Ba, J. (2014) ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION. [online]. Available from: https://arxiv.org/pdf/1412.6980.

Krishnapriya, S. & Karuna, Y. (2023) Pre-trained deep learning models for brain MRI image classification. Frontiers in Human Neuroscience. [Online] 17. [online]. Available from: https://www.frontiersin.org/journals/human-neuroscience/articles/10.3389/fnhum.2023.1150120/full (Accessed 16 February 2026).

Naser, M. A. & Deen, M. J. (2020) Brain tumor segmentation and grading of lower-grade glioma using deep learning in MRI images. Computers in Biology and Medicine. [Online] 121103758.

Purwono, P. et al. (2023) Understanding of Convolutional Neural Network (CNN): A Review. International Journal of Robotics and Control Systems. [Online] 2 (4), 739–748. [online]. Available from: https://www.researchgate.net/publication/367157330_Understanding_of_Convolutional_Neural_Network_CNN_A_Review (Accessed 18 February 2026).

Robbins, H. & Monro, S. (1951) A Stochastic Approximation Method. The Annals of Mathematical Statistics. [Online] 22 (3), 400–407. [online]. Available from: https://projecteuclid.org/journals/annals-of-mathematical-statistics/volume-22/issue-3/A-Stochastic-Approximation-Method/10.1214/aoms/1177729586.full (Accessed 19 February 2026).

Selvaraju, R. R. et al. (2019) Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. International Journal of Computer Vision. [Online] 128 (2), 336–359. [online]. Available from: https://arxiv.org/abs/1610.02391 (Accessed 22 February 2026).

Sharma, N. et al. (2018) An Analysis Of Convolutional Neural Networks For Image Classification. Procedia Computer Science. [Online] 132377–384. [online]. Available from: https://www.sciencedirect.com/science/article/pii/S1877050918309335 (Accessed 18 February 2026).

Shree Charran, R. & Dubey, R. K. (2022) Deep learning-based hybrid models for prediction of COVID-19 using chest X-ray. Novel AI and Data Science Advancements for Sustainability in the Era of COVID-19. [Online] 1–20. [online]. Available from: https://www.sciencedirect.com/science/chapter/edited-volume/pii/B9780323900546000015 (Accessed 19 February 2026).

Torrey, L. & Shavlik, J. (2010) Transfer Learning. [online]. Available from: https://ftp.cs.wisc.edu/machine-learning/shavlik-group/torrey.handbook09.pdf.

Warren, S. L. & Moustafa, A. A. (2022) Functional magnetic resonance imaging, deep learning, and Alzheimer's disease: A systematic review. Journal of Neuroimaging. [Online] 33 (1), 5–18. [online]. Available from: https://pmc.ncbi.nlm.nih.gov/articles/PMC10092597/ (Accessed 22 February 2026).

Python Code:

AIML_Assignment_Notebook

February 23, 2026

1 Dementia Classifier

This notebook is a record of my attempt to develop a machine learning model that predicts the disease status of an image. It contains mulitple methods and iterations until my final model.

Step 1. Load packages and unzip the images.

[1]:
```
# Import packages

# %pip install tensorflow
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import models, layers
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten,␣
↪Dropout, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
print(f"TensorFlow Version: {tf.__version__}")
print("Num GPUs Available: ", len(tf.config.list_physical_devices('GPU')))

#%pip install scikit-learn
from sklearn.metrics import classification_report, confusion_matrix, roc_curve,␣
↪auc, RocCurveDisplay
from sklearn.preprocessing import label_binarize

#%pip install seaborn
import seaborn as sns

import matplotlib.pyplot as plt

import os

import numpy as np

from collections import Counter

import glob
```

```
import os

import zipfile

import cv2

from google.colab import drive
drive.mount('/content/drive')
```

```
TensorFlow Version: 2.19.0
Num GPUs Available:  1
Mounted at /content/drive
```

1.1 Loading the images into the script.

[5]:
```
zip_path = '/content/drive/MyDrive/Colab Notebooks/images (1).zip'
extract_path = '/content/raw_data'

os.makedirs(extract_path, exist_ok=True)

# Unzips the folder into the raw_data folder.
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    for file in zip_ref.namelist():
        # Check if the file path contains the junk folders.
        if '__MACOSX' not in file and 'AD copy' not in file:
            zip_ref.extract(file, extract_path)

print("Data unzipped!")

# Verifies.
if os.path.exists(extract_path):
    print("Contents:", os.listdir(extract_path))
else:
    print("Folder not found!")
```

```
Data unzipped!
Contents: ['NoImpairment', 'ModerateImpairment', 'MildImpairment']
```

[6]:
```
basepath = '/content/raw_data'

# Our three classes
class_names = ['MildImpairment', 'ModerateImpairment', 'NoImpairment']
counts = []

# Loops through the classes list and creates the full path.
# Then outputs a list of all the items in the folder into 'files'.
```

```
# len() then counts the length of files, and the value is added to the counts␣
↪list.

for class_name in class_names:
    full_path = os.path.join(basepath, class_name)
    files = glob.glob(os.path.join(full_path, "*.jpg"))
    counts.append(len(files))

print (f"There is: {counts[2]} NoImpairment images.")
print (f"There is: {counts[0]} MildImpairment images.")
print (f"There is: {counts[1]} ModerateImpairment images.")
```

```
There is: 2560 NoImpairment images.
There is: 1792 MildImpairment images.
There is: 724 ModerateImpairment images.
```

[7]:
```
# Creates a bar chart of the distribution of data.
plt.bar(class_names, counts, color=['#4e79a7', '#f28e2b', '#e15759'])
plt.title ('Class Distribution')
plt.ylabel ('Number of Images')
plt.show()
```

1.2 Creating the datasets

[8]:
```
# Create the training dataset (80%)
train_ds = keras.utils.image_dataset_from_directory(
    basepath,
    image_size= (256,256),
    color_mode='grayscale',
    subset = "training",
    labels='inferred',
    validation_split=0.2,
    seed = 2614,
    shuffle = True
    )
# Creating the validation dataset (20%).
test_val_data = keras.utils.image_dataset_from_directory(
    basepath,
    image_size= (256, 256),
    color_mode='grayscale',
    labels='inferred',
    subset = "validation",
    validation_split=0.2,
    seed = 2614,
    shuffle = True)

# Split val/test
val_ds = test_val_data.take(len(test_val_data) // 2)
test_ds = test_val_data.skip(len(test_val_data) // 2)
print(f"Standard Datasets Ready: train_ds, val_ds, test_ds")
print(f"Shape: (256, 256, 1) - Grayscale")
```

```
Found 5076 files belonging to 3 classes.
Using 4061 files for training.
Found 5076 files belonging to 3 classes.
Using 1015 files for validation.
Standard Datasets Ready: train_ds, val_ds, test_ds
Shape: (256, 256, 1) - Grayscale
```

[9]:
```
# Stops training early if val_loss doesn't improve 4 epochs in a row, restores␣
↪back to the best epoch.
early_stopping_callback = EarlyStopping(
    monitor = 'val_loss',
    patience = 4,
    restore_best_weights = True
)
```

2 Model 1

[10]:
```
# A CNN with one layer.
modelv1 = models.Sequential([
    Input(shape=(256, 256, 1)), # 256 x 256 pixel images in greyscale
    Conv2D(filters=32, kernel_size=3, activation='relu'), # 32 feature maps at␣
↪a time, looking in a 3x3 window.
    MaxPooling2D (2,2), # Selects 4 pixels and retains the highest signal,␣
↪accelerates processing.
    Flatten(), # Turns it into a 1D array.
    Dropout(rate = 0.1), # Randomly turns off 10% of neurons: helps with␣
↪overfitting.
    Dense(units = 3, activation = 'softmax') # Final classification is one of␣
↪three classes.
])
```

[11]:
```
# Adam is a stochastic gradient descent optimisation algorithm
modelv1.compile(optimizer = Adam(learning_rate = 0.003),
              loss = "sparse_categorical_crossentropy",
              metrics =['accuracy'])
modelv1.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 254, 254, 32)	320
max_pooling2d (MaxPooling2D)	(None, 127, 127, 32)	0
flatten (Flatten)	(None, 516128)	0
dropout (Dropout)	(None, 516128)	0
dense (Dense)	(None, 3)	1,548,387

Total params: 1,548,707 (5.91 MB)

Trainable params: 1,548,707 (5.91 MB)

Non-trainable params: 0 (0.00 B)

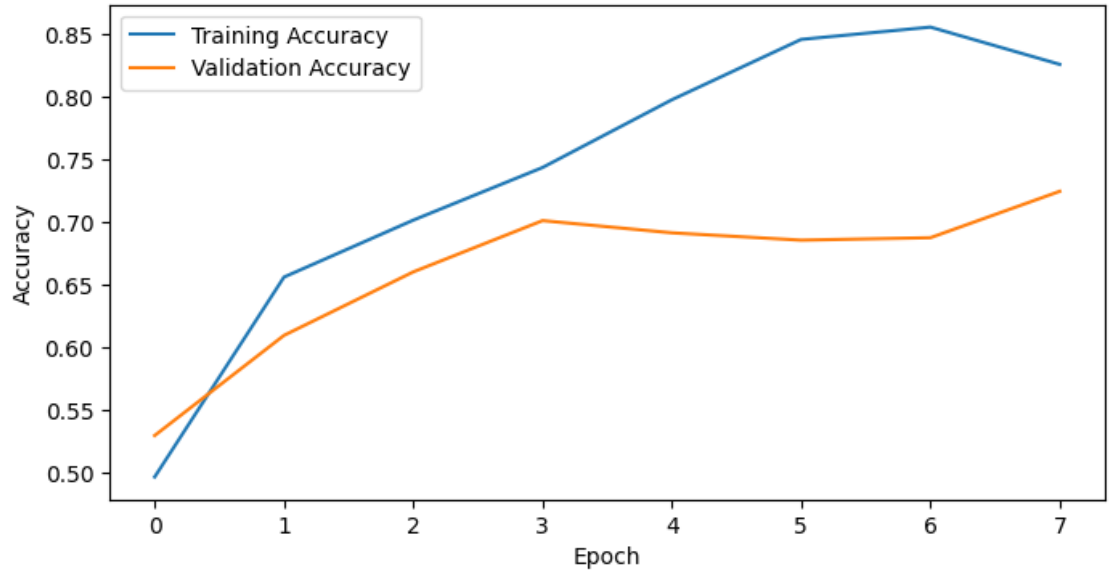
[12]:
```
# Trains the data on the training data set, while testing on the validation␣
↪dataset in between epochs.
historyv1 = modelv1.fit(train_ds, validation_data = val_ds,
    batch_size=32,
    epochs=20, shuffle=True,
    callbacks = [early_stopping_callback])
```

```
Epoch 1/20
127/127              12s 54ms/step -
accuracy: 0.4301 - loss: 1755.9559 - val_accuracy: 0.5293 - val_loss: 1.3316
Epoch 2/20
127/127              5s 36ms/step -
accuracy: 0.6657 - loss: 0.9130 - val_accuracy: 0.6094 - val_loss: 1.7226
Epoch 3/20
127/127              4s 31ms/step -
accuracy: 0.6451 - loss: 1.2145 - val_accuracy: 0.6602 - val_loss: 1.0877
Epoch 4/20
127/127              4s 35ms/step -
accuracy: 0.7469 - loss: 0.7016 - val_accuracy: 0.7012 - val_loss: 1.0265
Epoch 5/20
127/127              9s 68ms/step -
accuracy: 0.7738 - loss: 0.6652 - val_accuracy: 0.6914 - val_loss: 1.0279
Epoch 6/20
127/127              5s 40ms/step -
accuracy: 0.8530 - loss: 0.3860 - val_accuracy: 0.6855 - val_loss: 1.1173
Epoch 7/20
127/127              6s 47ms/step -
accuracy: 0.8447 - loss: 0.4383 - val_accuracy: 0.6875 - val_loss: 1.2661
Epoch 8/20
127/127              11s 50ms/step -
accuracy: 0.8198 - loss: 0.6007 - val_accuracy: 0.7246 - val_loss: 1.0617
```

2.1 Plotting accuracy and loss graphs for Model 1.

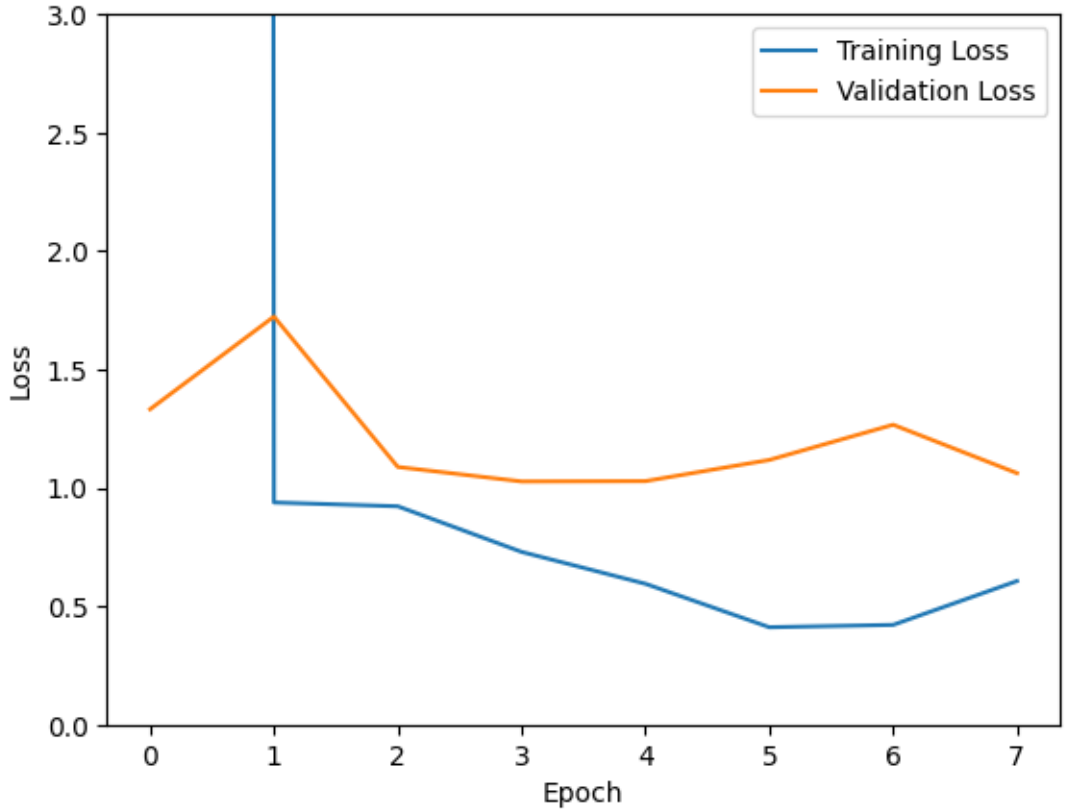
[13]:
```
# Plots training vs validation accuracy

epochv1 = historyv1.epoch

accv1 = historyv1.history['accuracy']
valaccv1 = historyv1.history['val_accuracy']

plt.figure(figsize=(8,4))
plt.plot(epochv1, accv1, label = 'Training Accuracy')
plt.plot(epochv1, valaccv1, label = 'Validation Accuracy')
plt.ylabel("Accuracy")
plt.xlabel("Epoch")
plt.legend()
```

```
plt.show()

# Provides the loss and accuracy on the test dataset.
loss, accuracy = modelv1.evaluate(test_ds)
print(f"Accuracy: {accuracy}\nLoss: {loss}")
```




```
16/16              2s 103ms/step -
accuracy: 0.6697 - loss: 1.1700
Accuracy: 0.6819085478782654
Loss: 1.0252161026000977
```

[14]:
```
# Plots training vs validation loss
lossv1 = historyv1.history['loss']
vallossv1 = historyv1.history['val_loss']

plt.plot (epochv1, lossv1, label = 'Training Loss')
plt.plot (epochv1, vallossv1, label = 'Validation Loss')
plt.ylabel("Loss")
plt.xlabel("Epoch")
plt.ylim (0,3)
plt.legend(loc = 'upper right')
plt.show()
```



2.2 Creating the classification report

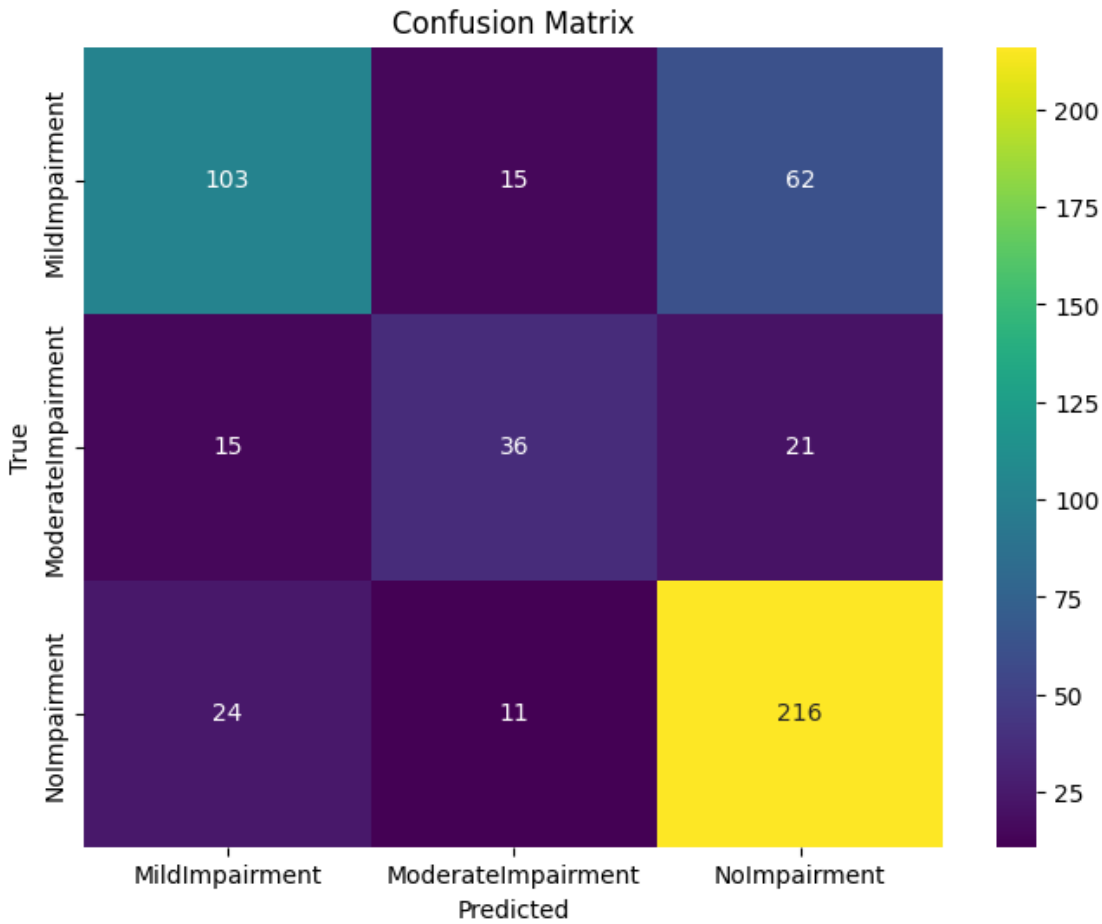
[15]:
```
# Create empty lists to hold the exact, locked order
y_true = []
x_test = []

# Extract both images and labels in a single loop
for images, labels in test_ds:
    x_test.extend(images.numpy())
    y_true.extend(labels.numpy())

# Convert to numpy arrays
x_test = np.array(x_test)
y_true = np.array(y_true)

# Generate predictions on the locked image array
y_prob = modelv1.predict(x_test)
y_pred = np.argmax(y_prob, axis=1)

# Print metrics
```

```
print(classification_report(y_true, y_pred, target_names=['MildImpairment',␣
↪'ModerateImpairment', 'NoImpairment']))
```

```
16/16              1s 25ms/step
                     precision    recall  f1-score   support

    MildImpairment       0.73      0.57      0.64       180
ModerateImpairment       0.58      0.50      0.54        72
      NoImpairment       0.72      0.86      0.79       251

          accuracy                           0.71       503
         macro avg       0.68      0.64      0.65       503
      weighted avg       0.70      0.71      0.70       503
```

2.3 Creating the confusion matrix

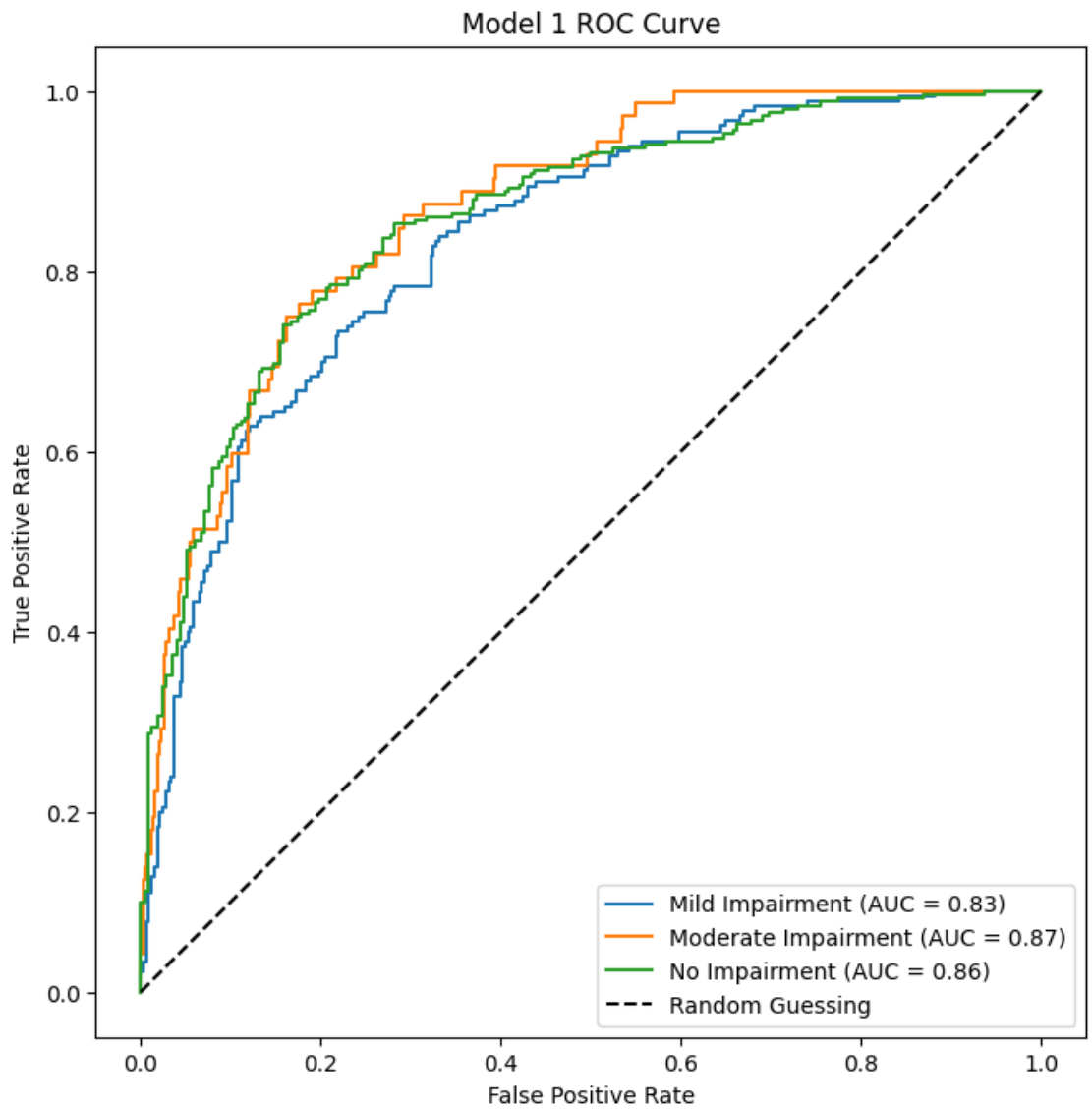
[16]:
```
# Create the confusion matrix
cm = confusion_matrix(y_true, y_pred)

# Plot it using a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='viridis',
            xticklabels=class_names,
            yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```



2.4 Creating a ROC Curve for Model 1

[32]:
```
fig, ax1 = plt.subplots(figsize=(8, 8))

for class_id in range(3):
    binary_labels = (y_true == class_id)
    class_probabilities = y_prob[:, class_id]

    RocCurveDisplay.from_predictions(
        binary_labels,
        class_probabilities,
        name=f"{class_names[class_id]}",
        ax=ax1
    )

ax1.plot([0, 1], [0, 1], "k--", label="Random Guessing")
ax1.axis("square")
```

```
ax1.set_xlabel("False Positive Rate")
ax1.set_ylabel("True Positive Rate")
ax1.set_title("Model 1 ROC Curve")
ax1.legend()
plt.show()
```



3 Second iteration of the model

The first model shows evidence of severe overfitting, so we do three things to address it:

- Data Augmentation

- Add multiple layers to the CNN
- Tune the learning rate

[17]:
```
# This model randomly flips, rotates and zooms each image to prevent␣
↪overfitting during training.
# Goes through 3 convolution layers, with filter doubling each time to learn␣
↪more complex features.
modelv2 = models.Sequential([
    Input(shape=(256, 256, 1)),
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.2),
    Conv2D(filters=32, kernel_size=3, activation='relu'),
    MaxPooling2D (2,2),
    Conv2D(filters=64, kernel_size=3, activation='relu'),
    MaxPooling2D (2,2),
    Conv2D(filters=128, kernel_size=3, activation='relu'),
    MaxPooling2D (2,2),
    Flatten(),
    Dense(units=128, activation='relu'),
    Dropout(rate = 0.3),
    Dense(units = 3, activation = 'softmax')
])

class_weight_dict = ({
    0 : 1.022,
    1 : 2.306,
    2 : 0.630
})
```

[18]:
```
modelv2.compile(optimizer = Adam(learning_rate = 0.0001),
              loss = "sparse_categorical_crossentropy",
              metrics =['accuracy'])
modelv2.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
random_flip (RandomFlip)	(None, 256, 256, 1)	0
random_rotation (RandomRotation)	(None, 256, 256, 1)	0
random_zoom (RandomZoom)	(None, 256, 256, 1)	0

```
conv2d_1 (Conv2D)                (None, 254, 254, 32)          320

max_pooling2d_1 (MaxPooling2D)   (None, 127, 127, 32)            0

conv2d_2 (Conv2D)                (None, 125, 125, 64)       18,496

max_pooling2d_2 (MaxPooling2D)   (None, 62, 62, 64)              0

conv2d_3 (Conv2D)                (None, 60, 60, 128)        73,856

max_pooling2d_3 (MaxPooling2D)   (None, 30, 30, 128)             0

flatten_1 (Flatten)              (None, 115200)                  0

dense_1 (Dense)                  (None, 128)            14,745,728

dropout_1 (Dropout)              (None, 128)                     0

dense_2 (Dense)                  (None, 3)                     387
```

Total params: 14,838,787 (56.61 MB)

Trainable params: 14,838,787 (56.61 MB)

Non-trainable params: 0 (0.00 B)

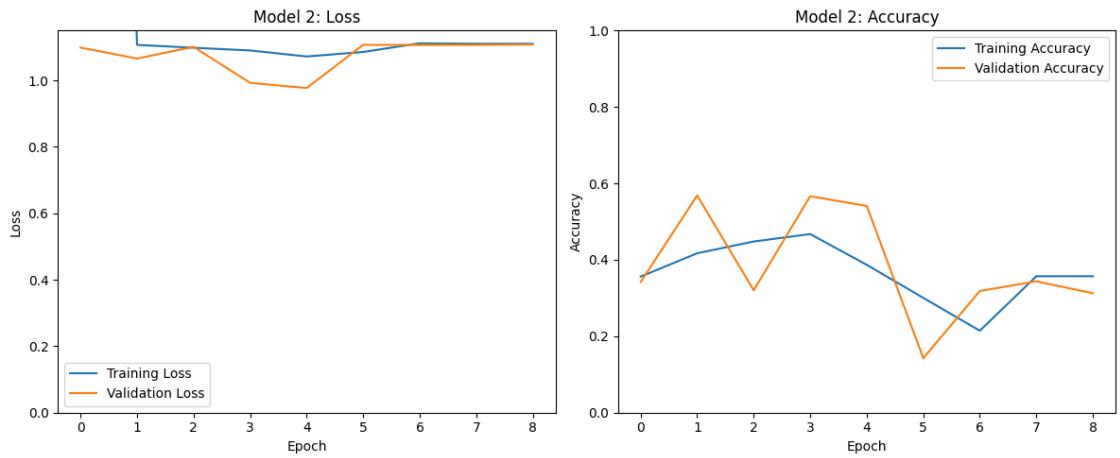
[19]:
```
historyv2 = modelv2.fit(train_ds, validation_data = val_ds,
batch_size=32,
epochs=20, shuffle=True,
class_weight = class_weight_dict,
callbacks = [early_stopping_callback])
```

```
Epoch 1/20
127/127              15s 75ms/step -
accuracy: 0.3538 - loss: 16.3327 - val_accuracy: 0.3418 - val_loss: 1.0988
Epoch 2/20
127/127              10s 78ms/step -
accuracy: 0.3934 - loss: 1.1200 - val_accuracy: 0.5684 - val_loss: 1.0657
Epoch 3/20
127/127              10s 77ms/step -
accuracy: 0.4276 - loss: 1.1131 - val_accuracy: 0.3203 - val_loss: 1.1010
Epoch 4/20
127/127              9s 68ms/step -
accuracy: 0.4521 - loss: 1.1039 - val_accuracy: 0.5664 - val_loss: 0.9926
```

```
Epoch 5/20
127/127              10s 69ms/step -
accuracy: 0.3781 - loss: 1.0931 - val_accuracy: 0.5410 - val_loss: 0.9771
Epoch 6/20
127/127              10s 67ms/step -
accuracy: 0.3525 - loss: 1.0837 - val_accuracy: 0.1426 - val_loss: 1.1070
Epoch 7/20
127/127              9s 67ms/step -
accuracy: 0.1653 - loss: 1.1187 - val_accuracy: 0.3184 - val_loss: 1.1066
Epoch 8/20
127/127              9s 70ms/step -
accuracy: 0.3583 - loss: 1.1278 - val_accuracy: 0.3438 - val_loss: 1.1067
Epoch 9/20
127/127              9s 68ms/step -
accuracy: 0.3675 - loss: 1.1179 - val_accuracy: 0.3125 - val_loss: 1.1079
```

[20]:

```python
# Extracting data for Model 2
epochv2 = historyv2.epoch
lossv2 = historyv2.history['loss']
vallossv2 = historyv2.history['val_loss']
accv2 = historyv2.history['accuracy']
valaccv2 = historyv2.history['val_accuracy']

# Create a 1x2 grid for side-by-side subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Plot 1: Model 2 Loss
ax1.plot(epochv2, lossv2, label='Training Loss')
ax1.plot(epochv2, vallossv2, label='Validation Loss')
ax1.set_title("Model 2: Loss")
ax1.set_ylabel("Loss")
ax1.set_xlabel("Epoch")
ax1.set_ylim(0, 1.15)
ax1.legend()

# Plot 2: Model 2 Accuracy
ax2.plot(epochv2, accv2, label='Training Accuracy')
ax2.plot(epochv2, valaccv2, label='Validation Accuracy')
ax2.set_title("Model 2: Accuracy")
ax2.set_ylabel("Accuracy")
ax2.set_xlabel("Epoch")
ax2.set_ylim(0, 1)
ax2.legend()

plt.tight_layout()
plt.show()
```



4 Creating the classification report

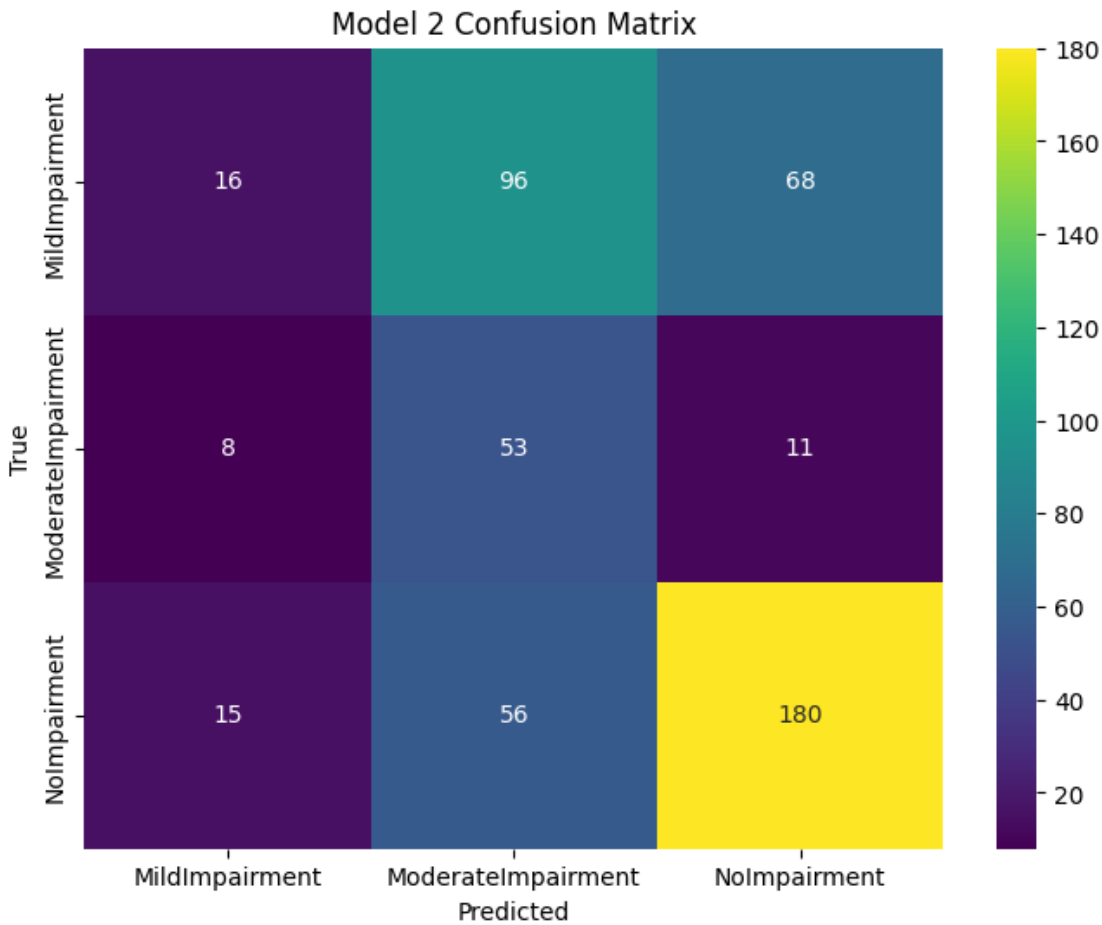
[21]:
```
y_prob_2 = modelv2.predict(x_test)
y_pred_2 = np.argmax(y_prob_2, axis=1)

print("--- Model 2 Classification Report ---")
print(classification_report(y_true, y_pred_2, target_names=['MildImpairment',␣
↪'ModerateImpairment', 'NoImpairment']))
```

```
16/16              2s 91ms/step
--- Model 2 Classification Report ---
                   precision    recall  f1-score   support

    MildImpairment       0.41      0.09      0.15       180
ModerateImpairment       0.26      0.74      0.38        72
      NoImpairment       0.69      0.72      0.71       251

          accuracy                           0.50       503
         macro avg       0.45      0.51      0.41       503
      weighted avg       0.53      0.50      0.46       503
```

5 Creating the confusion matrix

[22]:
```
# Create the confusion matrix
cm = confusion_matrix(y_true, y_pred_2)

# Plot it using a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='viridis',
```

```
            xticklabels=class_names,
            yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Model 2 Confusion Matrix')
plt.show()
```

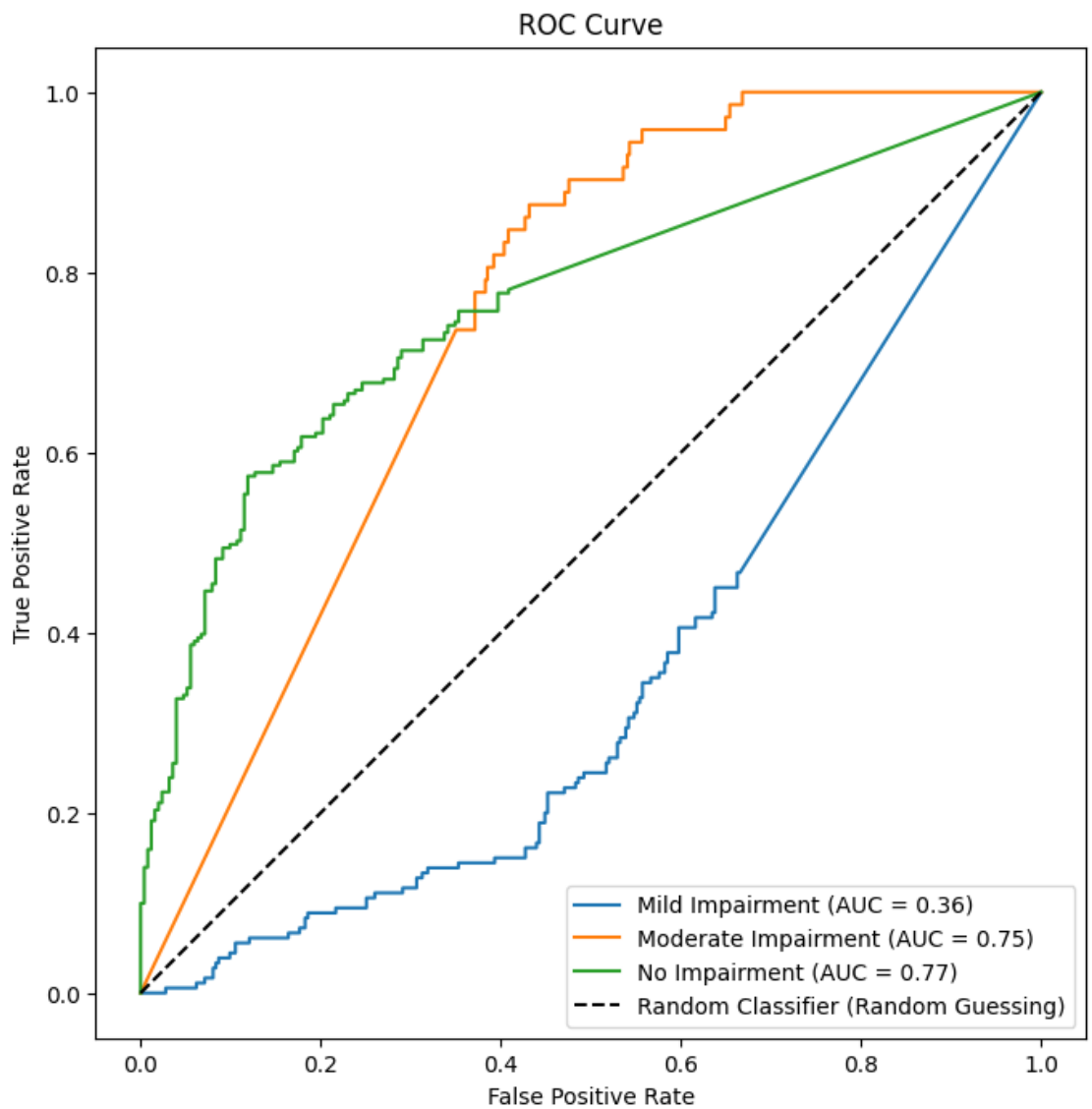


6 Creating the ROC Curve

[33]:
```
# Creates a ROC curve.
fig, ax = plt.subplots(figsize=(8,8))

# Loops through each class and generates a "One-vs-Rest" curve for each.
for class_id in range(3):
    binary_labels = (y_true == class_id) # Binarize the labels.
    class_probabilities = y_prob_2[:, class_id] # Select the probability column␣
 ↪corresponding to the current class.
```

```
    RocCurveDisplay.from_predictions( # Plot ROC curve
        binary_labels,
        class_probabilities,
        name=f"{class_names[class_id]}",
        ax=ax
    )

plt.plot([0,1], [0,1], "k--", label = "Random Classifier (Random Guessing)") #␣
 ↪Plots the diagonal line representing the random classifier.
plt.axis("square")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
```

[33]: <matplotlib.legend.Legend at 0x78725236ba10>



7 Model 3: Using ResNet

[26]:
```
# Load Base ResNet
base_resnet = keras.applications.ResNet50(
    include_top=False, # Removes the final classification layer
    weights='imagenet', # Uses the weights tuned from pre-trained images.
    input_shape=(224, 224, 3), # Standard ResNet shape
    pooling='avg' # Turn the 3D output into 1D vector, replaces Flatten()
)
base_resnet.trainable = False
```

```python
# Unfreezes the final 5 layers, except batch normalisations.
for layer in base_resnet.layers[-5:]:
    if "bn" not in layer.name:
        layer.trainable = True

modelv3 = models.Sequential([
    Input(shape=(256, 256, 1)),

    # Data Augmentation
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05), # 5% rotation

    # Adapter Block (Adjusts shape/color for ResNet)
    layers.Resizing(224, 224),
    layers.Lambda(lambda x: tf.image.grayscale_to_rgb(x)), # 1 Channel -> 3␣
 ↪Channels

    # Pre-trained Base
    base_resnet,

    # Classification layers at the end
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(3, activation='softmax')
])

modelv3.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss="sparse_categorical_crossentropy",
    metrics=['accuracy']
)

modelv3.summary()

class_weight_dict = ({
    0 : 1.022,
    1 : 2.306,
    2 : 0.630
})
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
94765736/94765736              5s
0us/step
```

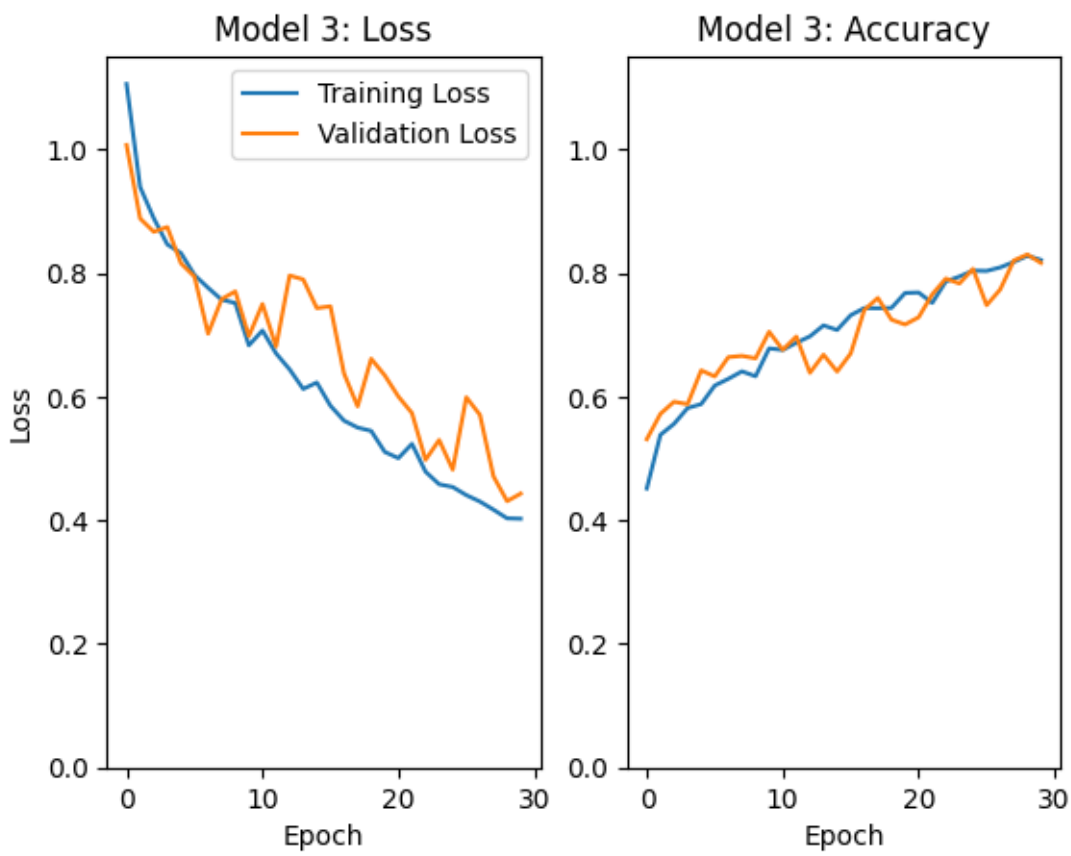
Model: "sequential_2"

```
Layer (type)                    Output Shape                  Param #

random_flip_1 (RandomFlip)      (None, 256, 256, 1)                0

random_rotation_1               (None, 256, 256, 1)                0
(RandomRotation)

resizing (Resizing)             (None, 224, 224, 1)                0

lambda (Lambda)                 (None, 224, 224, 3)                0

resnet50 (Functional)           (None, 2048)               23,587,712

dense_3 (Dense)                 (None, 128)                   262,272

dropout_2 (Dropout)             (None, 128)                        0

dense_4 (Dense)                 (None, 3)                        387


Total params: 23,850,371 (90.98 MB)


Trainable params: 1,313,283 (5.01 MB)


Non-trainable params: 22,537,088 (85.97 MB)
```

[ ]:
```
historyv3 = modelv3.fit(
    train_ds,
    validation_data=val_ds,
    epochs=30,
    class_weight = class_weight_dict,
    shuffle=True
)
```

[28]:
```
# Produces loss and accuracy graphs.
epochv3 = historyv3.epoch
lossv3 = historyv3.history['loss']
vallossv3 = historyv3.history['val_loss']
accv3 = historyv3.history['accuracy']
valaccv3 = historyv3.history ['val_accuracy']


fig, (ax1, ax2) = plt.subplots(1,2)
```

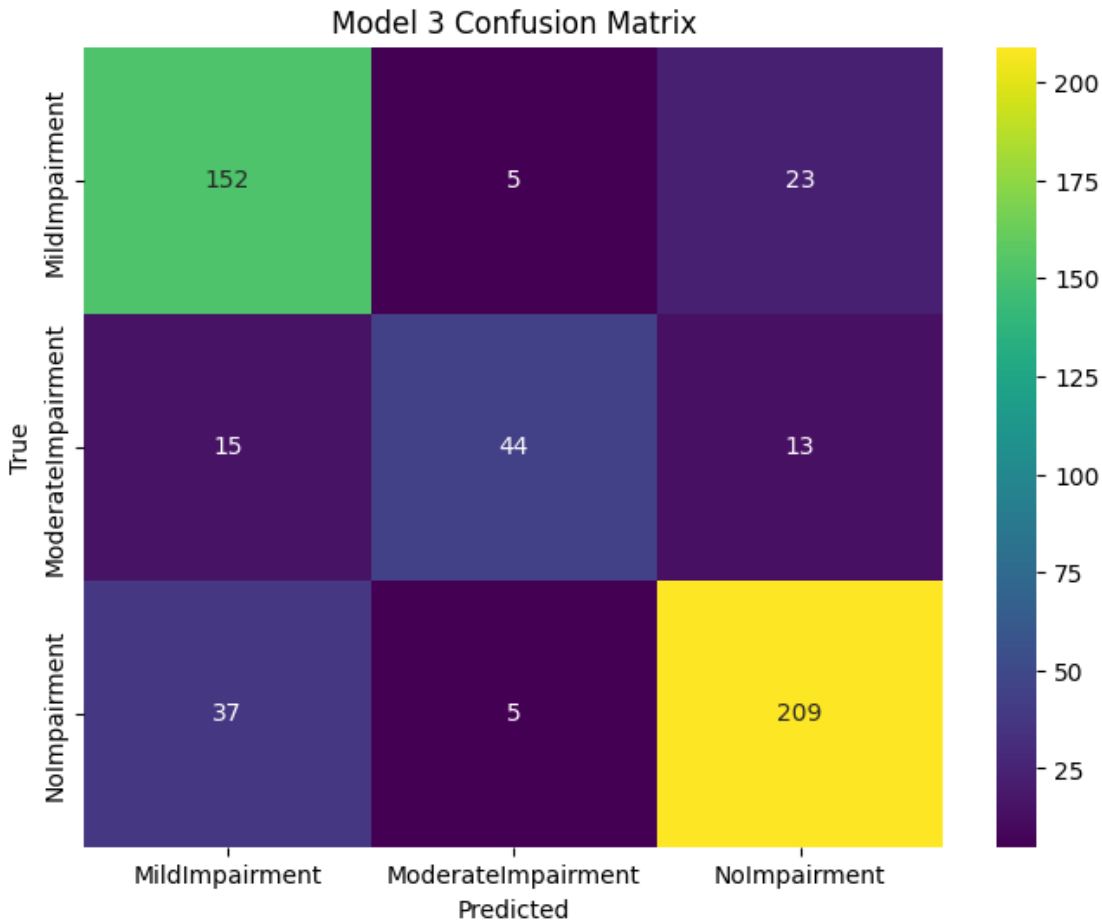
```
ax1.plot (epochv3, lossv3, label = 'Training Loss')
ax1.plot (epochv3, vallossv3, label = 'Validation Loss')
ax1.set_title("Model 3: Loss")
ax1.set_ylabel("Loss")
ax1.set_xlabel("Epoch")
ax1.set_ylim(0,1.15)
ax1.legend()

ax2.plot (epochv3, accv3, label = 'Training Accuracy')
ax2.plot (epochv3, valaccv3, label = 'Validation Training Accuracy')
ax2.set_title("Model 3: Accuracy")
ax2.set_ylim(0,1.15)
ax2.set_xlabel("Epoch")

plt.show()
```



8 Creating the classification table

[29]:
```
# Generate prediction probabilities
y_prob_3 = modelv3.predict(x_test, verbose=0)

# Convert probabilities to class labels
y_pred_3 = np.argmax(y_prob_3, axis=1)

# Print the final classification report
print("--- Model 3 Classification Report ---")
print(classification_report(y_true, y_pred_3, target_names=class_names))

# Create and plot the confusion matrix
cm_3 = confusion_matrix(y_true, y_pred_3)

plt.figure(figsize=(8, 6))
sns.heatmap(cm_3, annot=True, fmt='d', cmap='viridis',
            xticklabels=class_names,
            yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Model 3 Confusion Matrix')
plt.show()
```

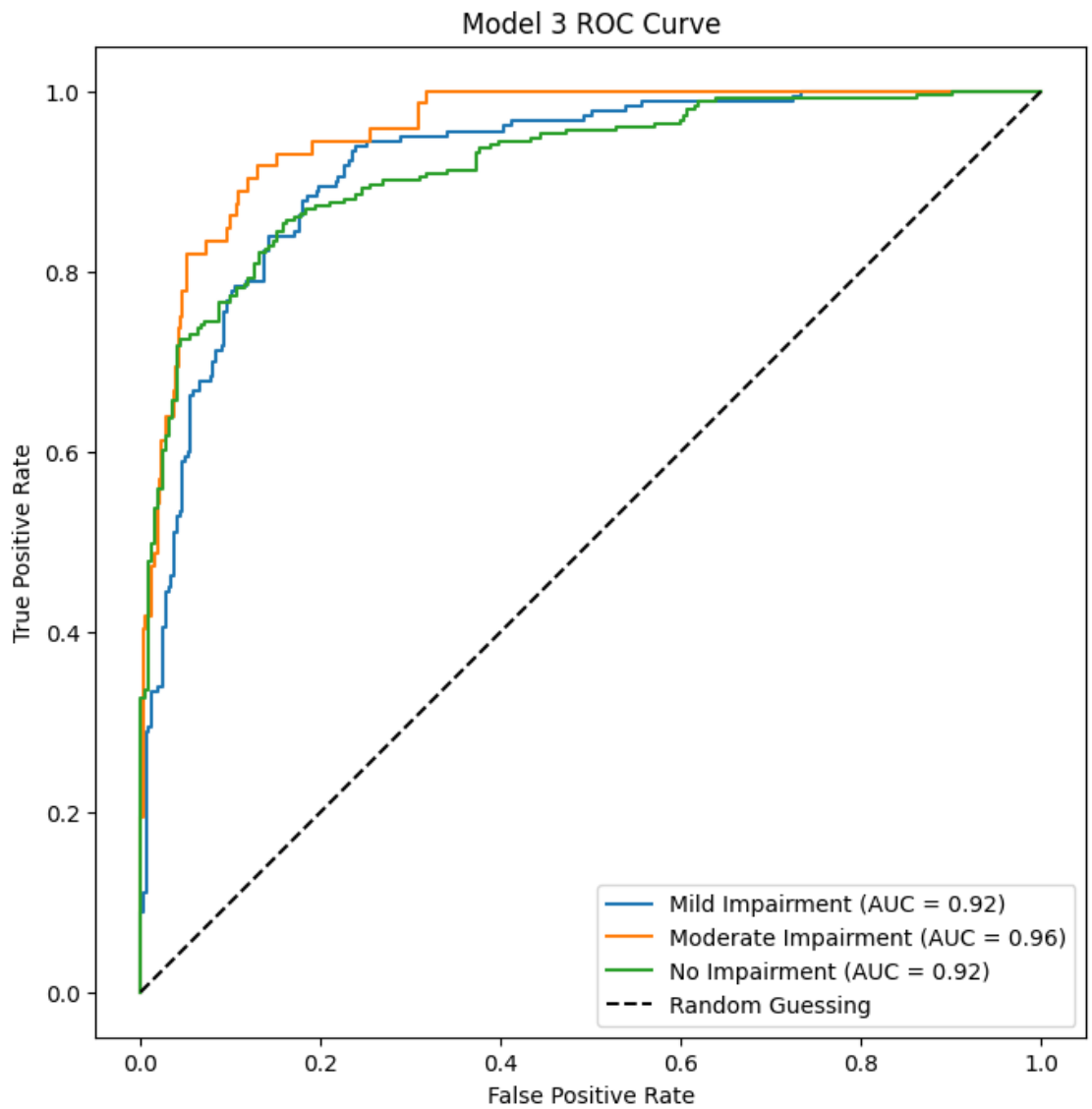
```
--- Model 3 Classification Report ---
                    precision    recall  f1-score   support

    MildImpairment       0.75      0.84      0.79       180
ModerateImpairment       0.81      0.61      0.70        72
      NoImpairment       0.85      0.83      0.84       251

          accuracy                           0.81       503
         macro avg       0.80      0.76      0.78       503
      weighted avg       0.81      0.81      0.80       503
```



9 Creating a ROC Curve for Model 3

[30]:
```
fig, ax = plt.subplots(figsize=(8, 8))

for class_id in range(3):
    binary_labels = (y_true == class_id)
    class_probabilities = y_prob_3[:, class_id]

    RocCurveDisplay.from_predictions(
        binary_labels,
        class_probabilities,
        name=f"{class_names[class_id]}",
        ax=ax
    )

plt.plot([0, 1], [0, 1], "k--", label="Random Guessing")
plt.axis("square")
```

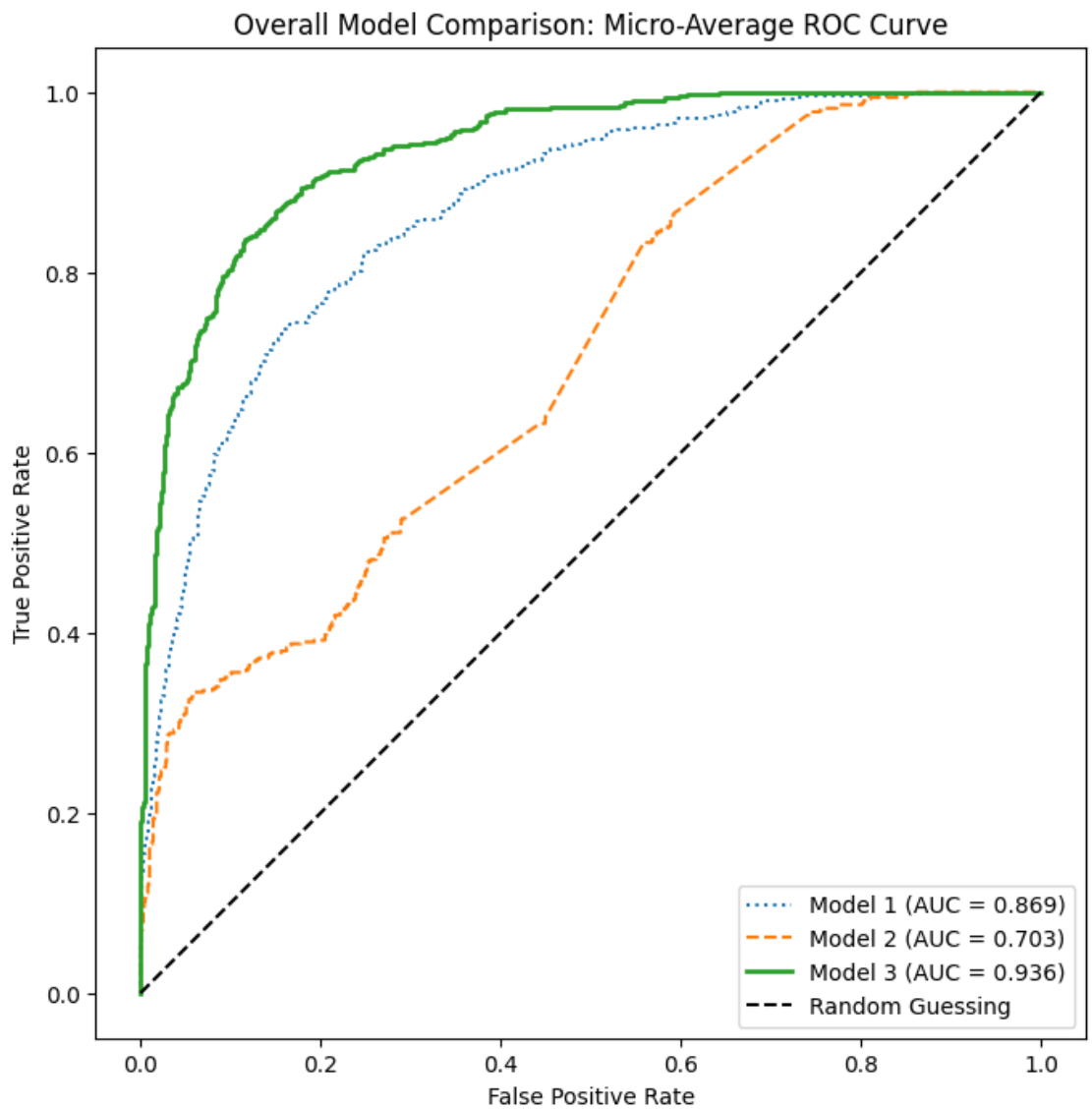
```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Model 3 ROC Curve")
plt.legend()
plt.show()
```



10 ROC Curve of all Three Models

[37]:
```
# Binarize true labels for multi-class micro-average computation
y_true_bin = label_binarize(y_true, classes=[0, 1, 2])
```

```python
# Helper function to compute micro-average ROC and AUC
def compute_micro_roc(y_true_b, y_prob):
    fpr, tpr, _ = roc_curve(y_true_b.ravel(), y_prob.ravel())
    roc_auc = auc(fpr, tpr)
    return fpr, tpr, roc_auc

# Calculate metrics for all three models
fpr1, tpr1, auc1 = compute_micro_roc(y_true_bin, y_prob)
fpr2, tpr2, auc2 = compute_micro_roc(y_true_bin, y_prob_2)
fpr3, tpr3, auc3 = compute_micro_roc(y_true_bin, y_prob_3)

# Plot the comparative ROC curves
plt.figure(figsize=(10, 8))
plt.plot(fpr1, tpr1, linestyle=':', label=f'Model 1 (AUC = {auc1:.3f})')
plt.plot(fpr2, tpr2, linestyle='--', label=f'Model 2 (AUC = {auc2:.3f})')
plt.plot(fpr3, tpr3, linewidth=2, label=f'Model 3 (AUC = {auc3:.3f})')

# Plot random guessing baseline
plt.plot([0, 1], [0, 1], 'k--', label='Random Guessing')

plt.axis('square')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Overall Model Comparison: Micro-Average ROC Curve')
plt.legend(loc='lower right')
plt.show()
```



[ ]:
```
# Saves the notebook as a PDF.
!sudo apt-get install texlive-xetex texlive-fonts-recommended␣
↪texlive-plain-generic
!jupyter nbconvert --to pdf '/content/drive/MyDrive/Colab Notebooks/
↪AIML_Assignment_Notebook.ipynb'
```

```
Reading package lists… Done
Building dependency tree… Done
Reading state information… Done
The following additional packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
  fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
  libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
```